

PE6201 End-of-Course Project - Trade-off Analysis Report

WANG KENAN (G2604062F) · Bank Customer-Ticket Triage Assistant

1. What I built, and how it changes things today

A back-office supervisor at a retail bank currently reads every incoming complaint ticket by hand to spot the urgent ones. My system takes a raw ticket and returns one structured flag - ESCALATE or MONITOR - so the human only reviews the flagged subset. In a simulated workload that was 2.5 human-hours per 100 tickets, the goal is to remove the routine reading and let the supervisor spend attention where a wrong call actually costs money: fraud, missing funds, locked accounts. The system is advisory only; a human always makes the final decision.

2. The technical choices and the trade-offs

Two classifiers, deliberately. I built the same task twice. A scikit-learn TF-IDF + logistic-regression model trained on 300 labelled tickets, and a rented gpt-4o-mini called with a plain prompt and no training. Both reached 1.000 accuracy on 100 held-out tickets - on this well-separated task, accuracy does not discriminate them.

The discrimination is on the other three axes, and that is the whole point:

```
| Metric | sklearn | LLM API |  
|---|---|---|  
| latency | 0.003 ms/ticket | 1.29 s/ticket |  
| marginal cost | ~$0 (local) | $0.000018/call |  
| effort to change a label | re-train | edit the prompt |
```

My build-vs-buy call follows the Class 2 stack: I rent the model and compute (gpt-4o-mini via OpenRouter) but own the data split, the prompt logic, and the L1/L2 eval harness. At current volume (a few thousand tickets/month) I would deploy the API - accuracy is already perfect and a new ticket type is a one-line prompt edit, not a re-labelling campaign. The choice flips around 100,000 tickets/month, where API cost and ~1 s latency dominate and a trained, on-prem, sub-millisecond model wins - at the price of a retraining pipeline. I rejected fine-tuning (no need for thousands of fresh labelled samples) and an agent (single-step classification needs no tool loop).

3. The RAG layer, and where it broke

I added a small retrieval component over 15 bank-policy documents. It answered five policy questions correctly (the 60-day fraud-reporting window, the \$20 archived-statement fee, repayment-holiday rules). But I deliberately broke it: the question `"I lost my card abroad - am I liable, and did I need to tell the bank before travel?"` spans two documents, and with top-k=1 the retriever only surfaced the lost-card policy. The model then confidently said the policy does not cover pre-travel notice - when that rule exists in a document it never retrieved. This was not a hallucination; it was faithful reasoning from an incomplete context. The fix is raising k or adding re-ranking, and the lesson is that a fluent, well-cited answer can still be wrong because retrieval was incomplete - so retrieval recall must be measured separately from answer quality.

4. The eval harness and the most interesting gap

My Part 3 harness scores every output two ways: L1 automatic assertions (valid JSON, valid label) and L2 judgement by a second LLM. A deliberately weak prompt scored L1 = 0% but L2 = 100%: the model made every triage decision correctly, but returned markdown prose instead of JSON, so no downstream system could parse it. Adding a JSON schema moved L1 to 100% with L2 unchanged. This is the inverse of the feared "L1 passes but L2 fails" trap - here format failed while substance was right. It shows the two levels catch different failures, and why you need both: eyeballing the output would have shipped v1 straight into an integration failure.

5. Critique: metrics, evals, and rough edges

I would not over-claim the 1.000 accuracy. My dataset is synthetic and the two classes are lexically separable - real tickets use rephrased grievances that blur the boundary, and a real evaluation must test on held-out `*production*` tickets, not templates. The LLM-as-judge is itself an unverified component; I did not calibrate it against my own hand labels, so a judge error could hide inside the L2 pass rate. Rough edges: the retriever uses plain TF-IDF rather than embeddings; k=1 is a known failure point; and I have not measured drift as ticket language evolves. On PDPA: the data is fully synthetic with no PII, so consent does not arise, but a production

swap-in of real tickets would need that review.

6. Future path (optional)

Next steps, in order: raise k and add re-ranking in the RAG; calibrate the LLM judge against hand labels; add an explicit ABSTAIN output for low-confidence tickets routed to humans; and re-test on real historical tickets before any pilot. The honest positioning is a triage aid under human supervision - not an autonomous decision system.